Informe de análisis y diseño Desafio final

Atlético Nacional y la gloria continental: Copa libertadores (2016)

Ronald Yesid Valencia Salazar

1. **Análisis del problema.**
   Al momento de realizar el analisis es fundamental entender a fondo que se estaba solicitando en la propuesta del videojuego. Ahi radica la pregunta: ¿Que elementos existen en la tematica de la copa libertadores de 2016 ganada por Atletico Nacional? existen jugadores, el balon con sus propiedades fisicas, la cancha, los rivales defensivos, el portero y las condiciones ambientales como la lluvia o la friccion del campo de juego. Cada uno de estos objetivos tiene sus propios datos y se relaciona con los demas. Este fue el punto de partida para delimitar la vision del juego y decidir las clases que tendríamos para realizar correctamente el videojuego.

   Contexto e inspiracion historica
   El videojuego recrea la historica copa libertadores ganada por Atletico Nacional en el año 2016, convirtiendose en el unico club colombiano en conseguir dos copas libertadores de america. El jugador tomara el control del equipo en momentos clave del torneo reviviendo el camino desde la fase de grupos hasta la gran final en el Atanasio Girardot.

   Caracteristicas y personalidad del protagonista
   Para darle personalidad al personaje principal y afectar dinamicamente su comportamiento dentro del juego se definen dos carcteristicas:
   - **Arriesgado / Aguerrido (Modo delantero):** Se activa cuando el jugador encadena pases exitosos o desbordes. Esta caracteristica le otorga un incremento del 30% en la potencia y curvatura de sus disparos a puerta, pero si realiza tiros continuos sin pausar, la presicion de apuntado disminuye temporalmente debido al cansancio.
   - **Veloz:** Al recibir un pase filtrado en el area rival, el personaje obtiene un impulso temporal de velocidad durante 4 segundos, lo que le permite pasar a los defensas rivales con mayor facilidad.

   Especificacion de los niveles.
   Para garantizar dinámicas de interaccion notablemente diferenciables, el videojuego cuenta con dos niveles con vistas y reglas distintas:
   - **Nivel 1: El camino a la gloria (Vista cenital / Top-Down)**
     - **Vista:** Cenital fija con desplazamiento de camara(Top-Down view)

    - **Sinopsis:** El jugador controla al armador en la mitad del campo. Debe elaborar la jugada rotando el balon, esquivando la presion alta de los defensas rivales y filtrando el pase gol.
    - **Reto y control por tiempo:** El nivel estaria regulado por un temporizador de 60 segundos. El objetivo es completar al menos 5 pases precisos y asistir al delantero antes de que el tiempo se acabe.
    - **Obstaculos:** Jugadores rivales que estan en zonas clave e interceptan pases.
- **Nivel 2: La noche coronada en el Atanasio(Vista lateral / Plataformero)**
    - **Vista:** Lateral fija / vista desde la perspectiva del arco o banda (side-view / platformer-style)
    - **Sipnosis:** Recreacion de las jugadas decisivas y cobros de tiro libre y centro en la final.
    - **Objetivo:** Ejecutar tiros libres o remates bombeados superando la barrera y venciendo al portero rival.
    - **Obstaculos:** La lluvia y el viento afectan la trayectoria del balon, sumando la reaccion del portero rival controlado por IA.

Modelos fisicos implementados

Se hicieron 4 modelos fisicos para simular el comportamiento realista del balon y el entorno:

1. Efecto Magnus(Curvatura lateral):
   Al realizar un disparo o pase con rosca, el balon experimenta una aceleracion lateral proporcional a la velocidad angular.
2. Rozamiento dinamico sobre el campo mojado:
   El balon pierde o mantiene velocidad al rodar segun el coeficiente de friccion del campo.
3. Movimiento parabolico(Tiros libres y bombeados):
   En tiros elevados, el balon sigue una trayectoria determinada por la gravedad y el angulo de tiro.
4. Movimiento armonico simple(Viento):
   El viento en la noche afecta la trayectoria del balon en el aire.

2. Agente inteligente(IA rival):

Para el control del portero y defensores rivales se piensa diseñar un agente simple con 4 fases independientes:

1. **Percepcion:** El agente persigue en tiempo real la posicion del balon, su velocidad y la postura del jugador.
2. **Razonamiento:** El agente calcula el punto probable de impacto del balon y decide la mejor accion(tirarse a la izquierda, derecha, centro o achicar el angulo de tiro).

3. **Accion:** Aplica el vector de fuerza y velocidad correspondiente al cuerpo del personaje y activa la animacion requerida(estirada, salto o bloqueo).
4. **Aprendizaje:** El agente almacena en memoria un historial con los ultimos disparos del usuario, si detecta alguna accion repetida (por ejemplo que el usuario dispara el 60% de las veces al poste derecho) ajusta las desiciones para anticiparse a esa zona en los siguientes intentos.

3. Propuesta de clases, atributos y metodos

- **Sugerencias de clases**
  - juego
  - jugador
  - balon
  - nivel
  - nivel1
  - nivel2
  - agente_inteligente
  - obstaculo_defensa

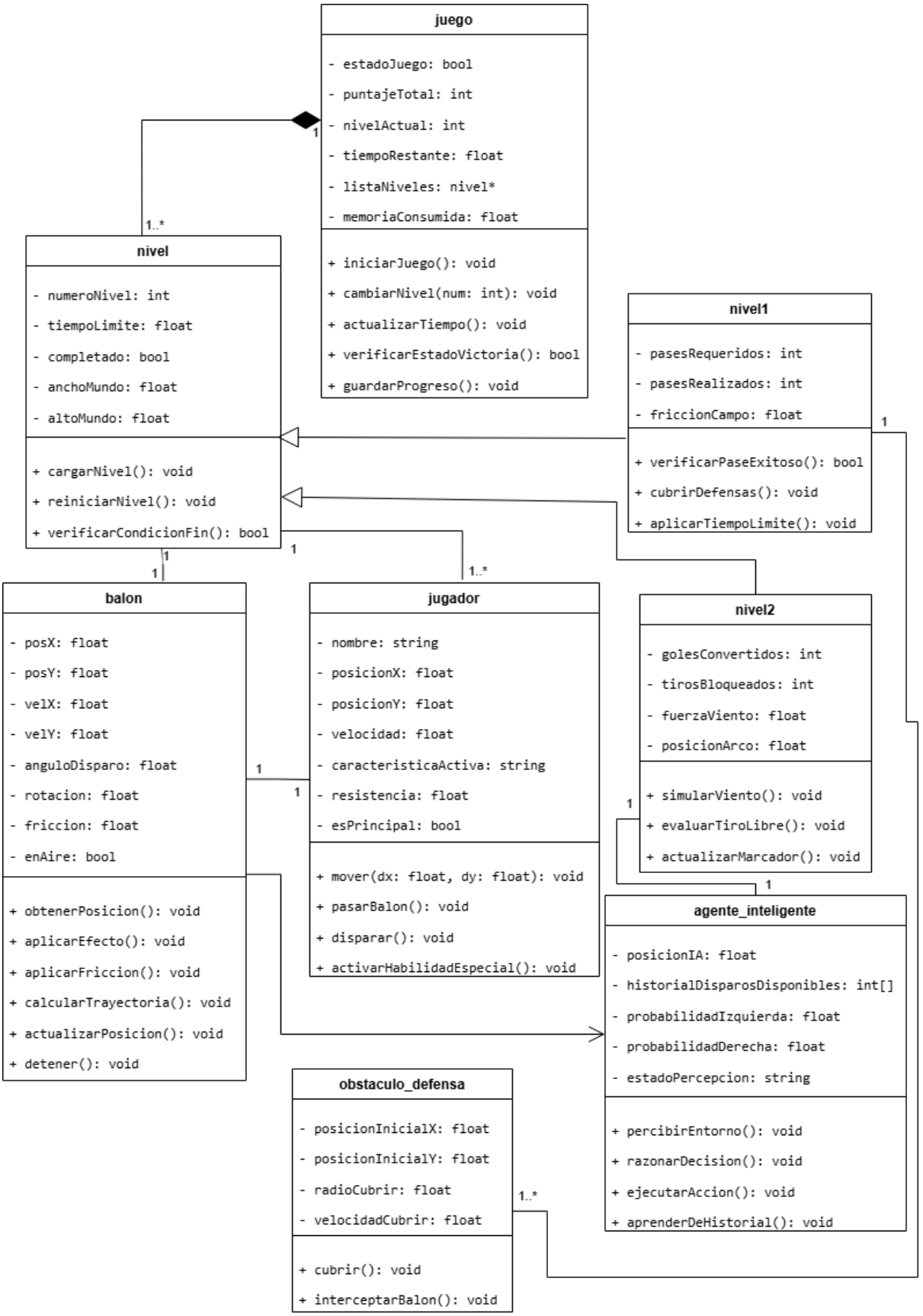
- **Sugerencias de atributos**
  - juego
    estadoJuego, puntajeTotal, nivelActual, tiempoRestante, listaNiveles, memoriaConsumida.
  - jugador
    nombre, posicionX, posicionY, velocidad, caracteristicaActiva, resistencia, esPrincipal.
  - balon
    posX, posY, velX, velY, anguloDisparo, rotacion, friccion, enAire.
  - nivel
    numeroNivel, tiempoLimite, completado, anchoMundo, altoMundo.
  - nivel1
    pasesRequeridos, pasesRealizados, defensasRival, friccionCampo.
  - nivel2
    golesConvertidos, tirosBloqueados, fuerzaViento, posicionArco.
  - agente_inteligente
    posicionIA, historialDisparosDisponibles, probabilidadIzquierda, probabilidadDerecha, estadoPercepcion.
  - obstaculo_defensa
    posicionInicialX, posicionInicialY, radioCubrir, velocidadCubrir.
- Sugerencias de metodos
  1. juego
     - iniciarJuego
     - cambiarNivel
     - actualizarTiempo
     - VerificarEstadoVictoria

- guardarProgreso

2. jugador
    - mover
    - pasarBalon
    - disparar
    - activarHabilidadEspecial
    - obtenerPosicion
3. balon
    - aplicarEfecto
    - aplicarFriccion
    - calcularTrayectoria
    - actualizarPosicion
    - detener
4. nivel
    - cargarNivel
    - reiniciarNivel
    - verificarCondicionFin
5. nivel1
    - verificarPaseExitoso
    - cubrirDefensas
    - aplicarTiempoLimite
6. nivel2
    - simularViento
    - evaluarTiroLibre
    - actualizarMarcador
7. agente_inteligente
    - percibirEntorno
    - razonarDecision
    - ejecutarAccion
    - aprenderDeHistorial
8. obstaculo_defensa
    - cubrir
    - interceptarBalon

Nota: En el apartado anterior se opto por agregar “sugerencias” ya que los datos e interfaces usados pueden estar sujetos a cambios al momento de realizar la implementación del programa dependiendo de las necesidades o dificultades que se presenten.

4. Diagrama UML



En la imagen anterior se presenta el diagrama de clases correspondiente a la capa logica del videojuego.